

Student Facility System

Security and Reliability Fixes Report

Prepared: 28 September 2026

Scope: authentication, payments, data integrity, uploads, frontend protection and operational safeguards

Executive summary

This report records the security and reliability improvements applied to the Student Facility System (SFS). The work prioritised account protection, prevention of duplicate charges, safe cancellation refunds, protected routes, controlled uploads, and safeguards against accidental data loss. Changes were designed to preserve existing application flows and to remain compatible with existing bearer-token API calls while introducing more secure session-cookie support.

1. Authentication and Sensitive Data Protection

- **Password and secret redaction**— Password hashes, reset-password codes, verification codes and their expiry values are removed from login, profile-update and email-verification responses. Password fields are also excluded from normal Mongoose queries by default. This prevents sensitive credentials being exposed to browsers or stored in session data.
- **Scoped, one-time password reset OTP**— Password reset OTP verification now matches the specific user ID, email and role from the initiating request. The OTP is consumed atomically when verified and the resulting reset token expires after 10 minutes. This prevents cross-account OTP use and replay attacks.
- **Enumeration resistance**— Forgot-password requests return the same public success message whether or not the email exists. This reduces an attacker's ability to discover registered accounts.
- **Ban and deletion enforcement**— Normal authenticated requests now verify that the account still exists and is not banned. Banned users are also blocked at login, so a previously issued JWT cannot continue working until it expires.
- **HTTP-only session support**— Login, verified registration and admin login now set an HTTP-only session cookie. Authentication middleware accepts this cookie while retaining bearer-header compatibility during migration. HTTP-only cookies reduce exposure to token theft through client-side scripts.

2. Payments, Booking and Refund Reliability

- **Atomic bed reservation**— Bed booking now uses an atomic conditional database update before creating a Stripe PaymentIntent. Only the first request can claim an available bed; concurrent requests receive a conflict response before a second card can be charged.
- **Reservation cleanup**— If Stripe declines a payment or fails to create an intent, the temporary bed reservation is released. This prevents failed payments from leaving beds unavailable.
- **Booking cancellation refunds**— The booking cancellation path attempts the Stripe refund before changing local booking or bed state. If the refund fails, the booking remains intact for a safe retry. Payment references are preserved for audit and reconciliation.

- **Kitchen order cancellation refunds**— When a kitchen owner cancels a paid order, SFS now requests a Stripe refund and records the refund/cancellation details. Invalid order-status transitions are rejected.

3. Data Integrity and Deletion Safeguards

- **Booked-bed protection**— Owners cannot delete a booked bed, renumber it, or mark it available through room editing. This protects a student's confirmed accommodation record.
- **Active-room protection**— A room containing pending, approved, booked or expiring reservations cannot be deleted.
- **Owner-deletion checks**— Deleting a hostel owner now removes dependent beds as well as rooms, but is blocked while active bookings exist. Kitchen-owner deletion is similarly blocked while active orders exist. This prevents orphaned operational records.
- **Safe hostel filtering**— User-provided facility text is escaped before it is used in a regular expression. This prevents malformed regex requests from causing server errors and reduces ReDoS exposure.

4. Upload and Frontend Access Controls

- **Authenticated image upload**— The Cloudinary image-upload endpoint now requires authentication. Upload type is restricted to an approved folder allow-list rather than accepting arbitrary Cloudinary folder paths. This protects storage quota and reduces abuse.
- **Protected routes**— A reusable ProtectedRoute component now blocks protected React screens from mounting before authentication/role checks. Student, hostel-owner, kitchen-owner and admin routes have role-based guards. This prevents protected-content flashes and unnecessary requests.
- **Client credential migration**— Axios is configured to send session cookies with cross-origin API requests. The standard login and post-verification flows no longer copy long-lived user tokens into JavaScript-readable cookies.

5. Operational Improvements and Validation

- **Chatbot cache**— Public chatbot responses are cached for 60 seconds with a 200-entry bound. Repeated questions no longer run the same live database aggregates repeatedly, reducing database load and improving response time.
- **Central request logging**— Server request logging now uses the existing logger utility, enabling production debug-log control and reducing accidental information leakage.
- **Automated test foundation**— A Jest test validates session-cookie parsing, including normal and absent-cookie cases. The test passed with 2/2 assertions.
- **Validation performed**— Changed backend files passed Node syntax checks; the targeted Jest session-cookie test passed; and git diff whitespace validation was performed.

6. Benefits Summary

- **Security**— Reduced account takeover, token exposure, user enumeration, upload abuse, banned-user access, regex injection and accidental disclosure of password hashes.

- **Financial safety**— Reduced risk of duplicate Stripe charges and ensured cancellations request refunds before changing local booking or order records.
- **Data quality**— Prevents deletion or modification of records that are still tied to active bookings or orders.
- **User experience**— Role guards prevent protected pages from briefly appearing to unauthorized visitors; caching makes repeated chatbot questions faster.

7. Recommended Follow-up Work

The following improvements remain recommended for a future hardening phase: migrate every remaining browser-readable legacy token reference to the HTTP-only session flow; add MongoDB transactions or compensating-write tests for every multi-document workflow; extend automated test coverage to authentication, booking races, refunds and deletion paths; and replace remaining raw console logging throughout controllers with the central logger. These are follow-up improvements and do not alter the implemented fixes described above.

End of report — Student Facility System (SFS)